

Object-Oriented Programming in R

Chapter 5

Stats 102A: Introduction to Computational Statistics with R

UCLA



Acknowledgements: Michael Tsiang and Miles Chen

(This chapter is largely drawn from Hadley Wickham's Advanced R, 2019)

- 1 Object-Oriented Programming
- 2 The S3 Object-Oriented System
- 3 Creating S3 Methods and Generics
- 4 Inheritance in S3 Classes

Required Packages

```
library(ggplot2)  
library(sloop)  
library(tidyverse)
```

Section 1

Object-Oriented Programming

Object-Oriented Programming

Object-oriented programming (OOP) is a style of programming that focuses on defining different types of objects and functions that can be applied to those object types.

Central to any object-oriented system are the concepts of class and method.

- A **class** is a definition of an object.
Typically a class contains several **fields** that are used to hold class-specific information.
- A **method** is a function for a specific class.

Intuitively, a class defines what an object is, and methods describe what that object can do.

Inheritance

Another common feature in object-oriented programming languages is the concept of **inheritance**. Classes are usually organized in a hierarchy, so if a method does not exist for a child, then its parent's method is used. We say the child **inherits** behavior from the parent.

For example, an ordered factor inherits from a regular factor, and a tibble inherits from a data frame.

Polymorphism

The primary reason to use OOP is **polymorphism** (which means “many shapes”), which refers to the ability to have a common interface for different types of inputs.

Polymorphic functions are ones which behave differently for different types of input objects.

Example of a Polymorphic Function in R

Consider the `summary()` function, which adapts its output depending on the type of input.

```
summary(diamonds$depth)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  43.00   61.00   61.80   61.75   62.50   79.00
```

```
summary(diamonds$cut)
```

```
##      Fair      Good Very Good   Premium      Ideal
##    1610     4906     12082     13791     21551
```

The most common polymorphic functions are `print()`, `summary()`, and `plot()`.

Three OOP Systems

Base R has three object-oriented systems (plus the base types).

- **S3** is R's first, simplest, and most flexible object-oriented system. S3 is the only object-oriented system used in the `base` and `stats` packages, and it is the most commonly used system in CRAN packages.
- **S4** is a formalization of S3 that has much stricter implementation for defining classes, methods, and generic functions. S4 is implemented in the `base` `methods` package.
- **R6** is a very different system from S3 and S4. R6 implements “message-passing” OOP, so methods belong to classes, not functions.

Base Types

There is one other system that is not quite object-oriented, but is important to mention:

- **Base types** are the internal C-level types that are the basis of the other object-oriented systems.

Base types do not form an OOP system, but they provide the building blocks for the other OOP systems.

Base Types

Underlying every R object is a C structure, or struct, that describes how that object is stored in memory. The struct includes the contents of the object, the information needed for memory management, and, most importantly for this section, a **type**. This is the **base type** of an R object.

Base types are not considered an object-oriented system because only the R core team can create new types. As a result, new base types are added very rarely.

We have already seen data structures, which cover the most common base types (the atomic vector types and lists). Base types also encompass functions, environments, and other more exotic objects.

You can determine an object's base type with `typeof()`.

The `is.object()` Function

Functions that behave differently for different base types are almost always written in C.

Even if you never write C code, it is important to understand base types, since everything else is built on top of them:

- S3 objects can be built on top of any base type
- S4 objects use a special base type
- R6 objects are a combination of S4 and environments (another base type).

The basic difference between base and object-oriented objects is that object-oriented objects have a **class attribute**.

To see if an object is a pure base type (i.e., does not have a class attribute), check that `is.object(x)` returns `FALSE`.

Section 2

The S3 Object-Oriented System

The S3 Object-Oriented System

S3 is R's first and simplest object-oriented system.

Most objects that you encounter are S3 objects. Unfortunately, there is no simple way to test if an object is an S3 object in base R.

The closest you can come is `is.object(x) & !isS4(x)`, i.e., it is an object but not an S4 object.

The `is.object()` Function

```
df <- data.frame(x = 1:10, y = letters[1:10])  
is.object(df)
```

```
## [1] TRUE
```

```
isS4(df)
```

```
## [1] FALSE
```

```
is.object(df$x)
```

```
## [1] FALSE
```

```
is.object(df$y)
```

```
## [1] FALSE
```

Defining Classes and Creating Objects

S3 is a simple and ad hoc system in the sense that it has no formal definition of a class.

To make an object an instance of a class, you only need to set the **class** attribute for a base object.

You can do that during creation with `structure()`, or after the fact with `class<-()`:

```
# Create and assign class in one step  
my_object <- structure(list(), class = "my_class")
```

```
# Create, then set class  
my_object <- list()  
class(my_object) <- "my_class"
```


The `class()` and `inherits()` Functions

S3 objects are usually built on top of lists or atomic vectors with attributes. Functions can also be turned into S3 objects.

You can determine the class of an S3 object with the `class()` function.

```
class(my_object)
```

```
## [1] "my_class"
```

You can see if an object inherits from a specific class using `inherits(x, "classname")`.

```
inherits(my_object, "my_class")
```

```
## [1] TRUE
```

Multiple Classes

The class of an S3 object can be a vector, which describes behavior from most to least specific.

For example, the class of the `tibble` object is a vector `c("tbl_df", "tbl", "data.frame")` indicating that tibbles inherit behavior from data frames.

```
class(diamonds)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

Ordered factors are also examples of objects with multiple classes.

```
class(diamonds$cut)
```

```
## [1] "ordered" "factor"
```

Constructors

Most S3 classes provide a **constructor** function that creates new objects with the correct structure.

The constructor should follow two principles:

- Have one argument for the base object, and one for each attribute.
- Check the type of the base object and the types of each attribute.

Constructor functions usually have the same name as the class, much like the built-in ones (e.g., `factor()` and `data.frame()`).

Constructors

An example of a constructor:

```
my_class <- function(x) {  
  if(!is.numeric(x)){  
    stop("x must be numeric")  
  }  
  structure(list(x), class = "my_class")  
}  
my_class(1:3)
```

```
## [[1]]  
## [1] 1 2 3  
##  
## attr("class")  
## [1] "my_class"
```

```
my_class("a")
```

Changing Classes

Aside from built-in constructor functions, S3 has no checks for correctness. This means you can change the class of existing objects:

```
# Create a linear model
lm_mtcars <- lm(log(mpg) ~ log(displacement), data = mtcars)
class(lm_mtcars)
```

```
## [1] "lm"
```

```
print(lm_mtcars)
```

```
##
## Call:
## lm(formula = log(mpg) ~ log(displacement), data = mtcars)
##
## Coefficients:
## (Intercept)      log(displacement)
##          5.3810          -0.4586
```

Changing Classes

```
# Change the class to data.frame
class(lm_mtcars) <- "data.frame"
print(lm_mtcars) # No longer works properly
```

```
## [1] coefficients residuals effects
## [4] rank fitted.values assign
## [7] qr df.residual xlevels
## [10] call terms model
## <0 rows> (or 0-length row.names)
```

```
# The data is still inside
lm_mtcars$coefficients
```

```
## (Intercept) log(displ)
## 5.3809725 -0.4585683
```

Changing Classes

The lack of built-in validation of classes has the potential to be problematic, but it rarely causes issues in practice.

While you *can* change the class of an object, you never should. R does not protect you from yourself.

Section 3

Creating S3 Methods and Generics

Generic Functions and Method Dispatch

In R, **generic functions**, or simply **generics**, are used to determine the appropriate method. The generic function determines the class of its argument(s) and uses that to select the appropriate method.

The process of finding the correct method given a class is called **method dispatch**.

The `print()`, `summary()`, and `plot()` functions are actually examples of generic functions.

Generic Function Example

The polymorphic behavior of generic functions allows you to use the same function name on different object types.

For example, the function `t()` can be used on a matrix:

```
x <- matrix(1:12, nrow = 4)
t(x)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12
```

Generic Function Example

The `t()` function can also be used on a data frame:

```
df <- data.frame(a = 1:4, b = 5:8, c = 9:12)
t(df)
```

```
##      [,1] [,2] [,3] [,4]
## a      1    2    3    4
## b      5    6    7    8
## c      9   10   11   12
```

Generic Functions and UseMethod()

In S3, methods belong to **generics**. S3 methods do not belong to objects or classes.¹

To determine if a function is an S3 generic, you can inspect its source code for a call to `UseMethod()`, which is the function that figures out the correct method to call, the process of **method dispatch**.

```
mean
```

```
## function (x, ...)
## UseMethod("mean")
## <bytecode: 0x11f8c7820>
## <environment: namespace:base>
```

¹Most OOP systems in other programming languages use **encapsulated OOP**, in which methods belong to objects/classes. S3 uses **functional OOP**, in which methods belong to functions.

Recognizing S3 Methods

Given a class, the job of an S3 generic is to call the right S3 method. You can recognize S3 methods by their names, which look like `generic.class()`.

For example, the `factor` method for the `summary()` generic is called `summary.factor()`.

```
ftype(summary)
```

```
## [1] "S3"      "generic"
```

```
ftype(summary.factor)
```

```
## [1] "S3"      "method"
```

Recognizing S3 Methods

The `generic.class()` method syntax is the reason that most modern style guides discourage the use of `.` in function names: it makes them look like S3 methods.

For example, is `t.test()` the `t` method for objects that are of class `test`?

Similarly, the use of `.` in class names can also be confusing: is `print.data.frame()` the `print()` method for `data.frame` class objects, or the `print.data()` method for `frame` class objects?

To reduce confusion, most style guides prefer the underscore `_` instead of the `.`, as in `read_csv()` or `as_tibble()`.

The methods() Function

You can see all the methods that belong to a generic with `methods()`:

```
methods("mean")
```

```
## [1] mean.Date          mean.default
## [3] mean.difftime      mean.POSIXct
## [5] mean.POSIXlt       mean.quosure*
## [7] mean.vctrs_vctr*
## see '?methods' for accessing help and source code
```

The methods() Function

You can also list all generics that have a method for a given class:

```
methods(class = "tbl_df")
```

```
##   [1] [               [[               [[<-
##   [4] [<-            %within%          $
##   [7] $<-            arrange_          as.data.frame
##  [10] coerce         distinct_          filter_
##  [13] fortify        group_data         initialize
##  [16] mutate_        names<-            nest_legacy
##  [19] nest           Ops                row.names<-
##  [22] show           slice_             slotsFromS3
##  [25] str            summarise_         tbl_sum
## see '?methods' for accessing help and source code
```


Method Dispatch Hierarchy

The `s3_dispatch()` function in the `sloop` package inputs a function call and outputs the list of all possible function names that are considered for method dispatch.

- The lack of a symbol means the method does not exist.
- The `=>` arrow means the method exists and is found by `UseMethod()`.
- The `->` arrow means the method exists and is used through `NextMethod()`.
- The `*` means the method exists but is not used.

Method Dispatch Hierarchy

For example, consider calling the generic `print()` function on an ordered factor:

```
s3_dispatch(print(factor(letters, ordered = TRUE)))
```

```
##      print.ordered  
## => print.factor  
## * print.default
```

There is no `print()` method for the ordered class, so the `print()` generic dispatches the `print.factor()` method for ordered objects.

Creating S3 Generics

The job of an S3 generic is to perform method dispatch, i.e. find the specific implementation for a class.

To create a new generic, create a function that calls `UseMethod()`.

The `UseMethod()` function takes two arguments: the name of the generic function, and the argument to use for method dispatch. The second argument defaults to the first argument of the function, which is usually what you want.

```
my_generic <- function(x){  
  UseMethod("my_generic")  
}
```

There is no need to pass any of the arguments of the generic to `UseMethod()`. The `UseMethod()` function will pass arguments to the method automatically (using what Hadley Wickham refers to as “deep magic”).

Creating S3 Methods

A generic is not useful without some methods.

To add a method to a generic, create a regular function with the correct (generic.class) name:

```
my_generic.my_class <- function(x){  
  "This is a my_class object!"  
}
```

```
my_object <- structure(list(c(1, 3)), class = "my_class")
```

```
class(my_object)
```

```
## [1] "my_class"
```

```
my_generic(my_object)
```

```
## [1] "This is a my_class object!"
```

Adding Methods to Existing Generics

Adding a method to an existing generic works in the same way:

```
mean.my_class <- function(x){  
  lapply(x, mean)  
}
```

```
mean(my_object)
```

```
## [[1]]
```

```
## [1] 2
```

As we saw previously with changing classes, there is no check to make sure that the method returns the class compatible with the generic. It is up to you to make sure that your method does not violate the expectations of existing code.

Method Dispatch

How does method dispatch work?

The `UseMethod()` function creates a vector of function names, like `str_c("generic", ".", c(class(x), "default"))` and looks for each potential method in turn.

The “**default**” class² makes it possible to set up a fallback method for otherwise unknown classes.

```
x <- factor("a", "b", "c")
str_c("generic", ".", c(class(x), "default"))
```

```
## [1] "generic.factor" "generic.default"
```

```
s3_dispatch(print(x))
```

```
## => print.factor
```

```
## * print.default
```

²The “default” class is a special **pseudo-class** in that it is not a real class, but we can define methods for it to use for inputs with unknown classes.

Method Dispatch Example

Consider the following generic and methods:

```
f <- function(x){  
  UseMethod("f")  
}
```

```
f.a <- function(x){  
  "Class a"  
}
```

```
f.default <- function(x){  
  "Unknown class"  
}
```

Method Dispatch Example

```
s3_dispatch(f(structure(list(), class = "a")))
```

```
## => f.a
```

```
## * f.default
```

```
f(structure(list(), class = "a"))
```

```
## [1] "Class a"
```

```
s3_dispatch(f(structure(list(), class = c("b", "a"))))
```

```
##      f.b
```

```
## => f.a
```

```
## * f.default
```

```
f(structure(list(), class = c("b", "a")))
```

```
## [1] "Class a"
```


Method Dispatch Example

```
s3_dispatch(f(structure(list(), class = "c")))
```

```
##      f.c
```

```
## => f.default
```

```
f(structure(list(), class = "c"))
```

```
## [1] "Unknown class"
```

Methods Are Functions

Because methods are normal R functions, they can also be called directly:

```
c <- structure(list(), class = "c")  
# Call the correct method:  
f.default(c)
```

```
## [1] "Unknown class"
```

```
# Force R to call the wrong method:  
f.a(c)
```

```
## [1] "Class a"
```

However, this is just as dangerous as changing the class of an object, so you should not do it.

Internal Generics

Some S3 generics, like `[`, `sum()`, and `cbind()`, do not call `UseMethod()` because they are implemented in C. Instead, they call the C functions `DispatchGroup()` or `DispatchOrEval()`.

Functions that do method dispatch in C code are called **internal generics** and are documented in `?"internal generic"`.

Section 4

Inheritance in S3 Classes

Inheritance in S3 Classes

S3 classes can share behavior through **inheritance**.

Inheritance in S3 follows three principles:

- The class of an object can be a character vector.
- If a method is not found for the class in the first element of the vector, R looks for a method for the second class (and so on).
- A method can delegate work by calling `NextMethod()`.

We will expand on these principles in this section.

Subclasses and Superclasses

The ordered class is said to be a **subclass** of factor because it always appears before it in the class vector.

Conversely, the factor class is a **superclass** of ordered.

S3 has no formal restrictions on the relationship between sub- and superclasses, but there are two principles to follow when creating a subclass:

- The base type of the subclass should be that same as the superclass.
- The attributes of the subclass should be a superset of the attributes of the superclass.

Allowing Subclassing

When you create a class, you need to decide if you want to allow subclasses, because it requires some changes to the constructor and careful thought in your methods.

To allow subclasses, the parent constructor needs to have `...` and `class` arguments:

```
digits <- function(x, ..., class = character()){  
  if(!is.numeric(x)){  
    stop("x must be numeric")  
  }  
  structure(x, class = c(class, "digits"))  
}
```

Subclass Constructor

Then the subclass constructor can just call to the parent class constructor with additional arguments as needed.

```
my_digits <- function(x) {  
  digits(x, class = "my_digits")  
}
```

```
y <- my_digits(c(1, 311, 198))
```

```
class(y)
```

```
## [1] "my_digits" "digits"
```


The Inherited Method

Suppose we create a `print.digits` method that returns the number of digits:

```
print.digits <- function(x){  
  print(nchar(x))  
}
```

```
x <- digits(c(24601, 8675309, 102))
```

```
x
```

```
## [1] 5 7 3
```

```
y
```

```
## [1] 1 3 3
```

The `NextMethod()` Function

For subclasses, the method for a subclass can dispatch the method of a superclass (usually the next class in the class vector) instead of its own class using `NextMethod()`. This allows a subclass to use a function that has been previously written and pass arguments to it, rather than having to write a new method from scratch.

The **next method** is defined to be the method which would have been called if the current method did not exist. In other words, the next method is the first inherited method.

The `NextMethod()` function calls the next method.

Example of NextMethod()

We have created a `print.digits` method that returns the number of digits:

```
print.digits <- function(x){  
  print(nchar(x))  
}  
  
x <- digits(c(24601, 8675309, 102))  
x
```

```
## [1] 5 7 3
```

Example of NextMethod()

```
x

## [1] 5 7 3

print.my_digits <- function(x){
  print(str_extract(unclass(x), "\\d+"))
  NextMethod()
}
```

Note: The syntax for `NextMethod()` is similar to `UseMethod()` in the sense that no arguments need to be explicitly passed. The `NextMethod()` function will find and pass all arguments automatically.

```
y
```

```
## [1] "1"    "311" "198"
```

```
## [1] 1 3 3
```

The actual method dispatch from `print(y)` is shown here:

```
s3_dispatch(print(y))
```

```
## => print.my_digits
```

```
## -> print.digits
```

```
## * print.default
```

We see that the `print.my_digits` method exists, but the `print.digits` method was also the one called through `NextMethod()`.